Tema 4: Socket
Trabajo Práctico

Entorno de trabajo

- Sistema operativo: Linux (Ubuntu Server recomendado).
- Lenguaje: C, usando solo la biblioteca estándar y las cabeceras POSIX.
- Paquetes necesarios: `sudo apt install build-essential netcat-openbsd tmux`.

Compilación

Todos los programas deben compilar sin errores ni advertencias con:

```
gcc -Wall -Wextra -o programa programa.c
```

Herramientas de prueba

- `nc` (netcat): cliente o servidor TCP/UDP genérico, para probar un extremo sin programar el otro.
- `ss -tan`: muestra las conexiones TCP y su estado.
- `tmux` o varias terminales: para ejecutar servidor y cliente al mismo tiempo.

Requisitos comunes a todos los programas

1. Verificar el valor de retorno de todas las llamadas a la API e informar los errores con `perror()`, `strerror()` o `gai_strerror()`, según corresponda.
2. Validar los argumentos de línea de comandos y mostrar un mensaje de uso si son incorrectos.
3. Aplicar las conversiones de orden de bytes (`htons`, `htonl`, `ntohs`, `ntohl`) en todos los valores que se escriban o lean de una estructura de dirección.
4. Inicializar en cero las estructuras de dirección antes de completarlas.
5. Liberar los recursos: cerrar los descriptores y liberar las listas de `getaddrinfo()`.
6. Comentar el código explicando las decisiones importantes.

Ejercicio 1

Objetivo: comprender que devuelve la función socket() y cómo se detectan e informan sus errores.

Consigna: escribir el programa ej01_sockets.c que:

1. Cree, sin cerrarlos todavía, tres sockets: TCP sobre IPv4, UDP sobre IPv4 y TCP sobre IPv6, usando 0 como protocolo.
2. Muestre el descriptor obtenido para cada uno.
3. Cierre los tres sockets.
4. Intente crear un socket con la combinación socket(AF_INET, SOCK_STREAM, IPPROTO_UDP) y muestre el error obtenido.

Si la creación de un socket falla, el programa debe informar y continuar con los demás.

Preguntas:

1. ¿Qué valores tienen los descriptores obtenidos? ¿Por qué son esos y no otros?
2. ¿Qué sucede con la numeración si se cierra el primer socket y se crea uno nuevo? Comprobarlo modificando el programa.
3. ¿Qué error se obtiene en el punto 4? Explicar por qué esa combinación no es válida, relacionándolo con la diferencia entre tipo de socket y protocolo.
4. ¿Qué error se obtiene con socket(AF_INET, SOCK_DGRAM, IPPROTO_TCP)? ¿Por qué?

Ejercicio 2

Objetivo: observar en memoria la diferencia entre el orden de bytes del host y el de la red, y el efecto de las funciones de conversión.

Consigna: escribir el programa ej02_endianness.c que:

1. Determine si la máquina es *little-endian* o *big-endian*, examinando el primer byte en memoria de un uint16_t con valor 0x0102.
2. Muestre byte por byte, en hexadecimal, el número 8080 antes y después de aplicarle htons().
3. Muestre byte por byte el valor de 32 bits 0xC0A8010A (la dirección 192.168.1.10) antes y después de aplicarle htonl().
4. Asigne el valor 8080 al campo sin_port de un struct sockaddr_in sin convertirlo y muestre el resultado de aplicarle ntohs(). Repetir asignándolo correctamente con htons().

Sugerencia: para examinar los bytes de una variable, recorrerla con un puntero a unsigned char.

Preguntas:

1. ¿Es la máquina *little-endian* o *big-endian*? Justificar a partir de la salida del programa.
2. ¿Qué valor devuelve ntohs() en el punto 4 cuando no se usa htons()?
3. ¿Qué consecuencia tendría ese error en un servidor real?

Ejercicio 3

Objetivo: usar getaddrinfo() y getnameinfo() para traducir nombres de host y servicios a direcciones y viceversa.

Consigna: escribir el programa ej03_resolver.c, que se ejecuta como ./ej03_resolver <host> <servicio> y:

1. Llame a getaddrinfo() solicitando direcciones de cualquier familia (AF_UNSPEC) para sockets de tipo SOCK_STREAM.
2. Recorra la lista devuelta y, para cada nodo, muestre la familia (IPv4 o IPv6), la dirección IP en texto y el número de puerto.
3. Para cada dirección, obtenga su nombre inverso con getnameinfo(). Si no existe, debe indicarlo.
4. Libere la lista con freeaddrinfo().
5. Informe los errores de resolución con gai_strerror().

Pruebas mínimas: localhost http, google.com https, example.com 8080 y un nombre inexistente.
Preguntas:

1. ¿Cuántas direcciones devuelve cada nombre? ¿Por qué un mismo nombre puede tener varias?
2. ¿De dónde obtiene el sistema el número de puerto del servicio http?
3. ¿Por qué los errores de getaddrinfo() no se informan con perror()?
4. ¿Qué ventajas tiene getaddrinfo() sobre gethostbyname()?

Ejercicio 4

Objetivo: implementar la secuencia básica de un servidor TCP: socket(), bind(), listen() y accept().
Consigna: escribir el programa ej04_saludo_servidor.c, que se ejecuta como ./ej04_saludo_servidor <puerto> y:

1. Cree un socket TCP y lo asocie a la dirección comodín y al puerto indicado.
2. Por cada cliente que se conecte, muestre en pantalla su dirección IP y su puerto.
3. Le envíe al cliente el mensaje Hola, te conectaste desde <IP>:<puerto>.
4. Cierre la conexión con ese cliente y quede esperando al siguiente, sin terminar.

Prueba: ejecutar el servidor y conectarse varias veces seguidas con nc localhost <puerto>.
Preguntas:

1. ¿Qué se observa en el puerto del cliente en cada conexión? ¿Quién lo asigna y por qué el cliente no llama a bind()?
2. ¿Qué diferencia hay entre el descriptor que devuelve socket() y el que devuelve accept()? ¿Cuál se cierra después de atender a cada cliente?

Ejercicio 5

Objetivo: implementar un cliente y un servidor TCP completos y observar el estado TIME_WAIT del cierre de conexión.
Consigna: escribir dos programas.
ej05_eco_servidor.c (./ej05_eco_servidor <puerto>):

1. Atiende a los clientes de a uno por vez.
2. Devuelve al cliente todo lo que recibe, hasta que este cierre la conexión.
3. Muestra cuándo se conecta y cuándo se desconecta cada cliente.
4. Debe garantizar que se envían todos los bytes recibidos, aunque send() envíe menos de lo pedido.

5. Debe aceptar un segundo argumento opcional, noreuse, que desactiva la opción SO_REUSEADDR.

ej05_eco_cliente.c (./ej05_eco_cliente <host> <servicio>):

1. Resuelve el destino con getaddrinfo() y prueba cada dirección de la lista hasta que una conexión tenga éxito, mostrando cada intento y su resultado.
2. Lee líneas del teclado, las envía al servidor y muestra la respuesta completa.
3. Termina cuando se ingresa fin de entrada (Ctrl+D).

Experimento:

1. Con un cliente conectado, terminar el servidor con Ctrl+C. Observar el estado de la conexión con ss -tan.
2. Reiniciar inmediatamente el servidor con la opción noreuse y observar el resultado.
3. Repetir el reinicio sin la opción noreuse.
4. Repetir el experimento cerrando primero el cliente.
5. Ejecutar el cliente contra un puerto donde no hay ningún servidor.

Preguntas:

1. ¿En qué estado queda la conexión en el paso 1 y en qué extremo? ¿Cuánto dura ese estado y para qué sirve?
2. ¿Qué error se obtiene en el paso 2 y por qué no ocurre en el paso 3?
3. ¿Dónde aparece el estado TIME_WAIT en el paso 4? ¿Qué regla general se deduce?
4. ¿Qué error se obtiene en el paso 5? ¿Qué indica sobre el destino?
5. ¿Por qué el cliente no debe limitarse a la primera dirección que devuelve getaddrinfo()?

Ejercicio 6

Objetivo: programar con sockets de datagramas y compensar la falta de confiabilidad de UDP desde la aplicación.

Consigna: escribir dos programas.

ej06_udp_servidor.c (./ej06_udp_servidor <puerto>): recibe datagramas con recvfrom(), muestra la dirección de origen y el tamaño de cada uno, y los devuelve al remitente con sendto().

ej06_udp_cliente.c (./ej06_udp_cliente <host> <puerto> [conectado]):

1. Envía tres datagramas de tamaños distintos (por ejemplo, 1, 18 y 1000 bytes) y verifica que cada respuesta tenga exactamente el tamaño enviado.
2. Espera cada respuesta como máximo 2 segundos, usando la opción SO_RCVTIMEO.
3. Si no llega respuesta, reintentar hasta 3 veces antes de abandonar ese mensaje, informando cada intento.
4. Sin el argumento conectado, usa sendto() y recvfrom(). Con el argumento, usa connect() y luego send() y recv().

Experimento: con el servidor apagado, ejecutar el cliente en sus dos modos y comparar los resultados.

Preguntas:

1. ¿Cada recvfrom() devuelve exactamente un datagrama? ¿En qué se diferencia esto de TCP?
2. ¿Qué se observa en cada modo con el servidor apagado? ¿Por qué solo uno de ellos detecta que no hay servidor?

3. ¿Qué hace connect() sobre un socket UDP? ¿Envía algún paquete?
4. Aunque se use el modo conectado, ¿por qué sigue siendo necesario el timeout?